



Final Exam, Theory of Computation 2022

- Books, notes, communication, calculators, cell phones, computers, etc... are not allowed.
- Your explanations and proofs should be clear enough and in sufficient detail so that they are easy to understand and have no ambiguities.
- You are allowed to refer to material covered in lectures (but not exercises) including theorems without reproving them.
- **Do not touch until the start of the exam.**

Good luck!

Name: _____

N° Sciper: _____

Problem 1	Problem 2	Problem 3	Problem 4	Problem 5	Problem 6
/ 15 points	/ 15 points	/ 15 points	/ 20 points	/ 20 points	/ 15 points

Total / 100

1 (15 pts) **Quick-fire round.** Consider the following statements.

1. If A is regular and $B \leq_p A$, then B is regular.
2. If A can be recognised by an NFA, then $\overline{A}$ can also be recognised by an NFA.
3. If A is decidable and $B \subseteq A$, then B is decidable.
4. If A is **NP**-complete and $A \leq_p B$, then B is **NP**-complete.
5. If A is recognisable and $A \leq_m \overline{A}$, then A is undecidable.
6. If A and B are recognisable, then $A \cap B$ is recognisable.
7. The language $\{0^n 1^m : n - m \text{ is divisible by } 2022\}$ is regular.
8. The language $\{\langle D \rangle : D \text{ is a DFA and } L(D) \text{ is infinite}\}$ is decidable.
9. The language $\{\langle M \rangle : M \text{ is a TM and } L(M) \text{ is infinite}\}$ is recognisable.
10. The language $\{\langle G \rangle : G \text{ is a graph that contains a clique of size } 2022\}$ is in **P**.
11. $3\text{-SAT} \leq_p 2\text{-SAT}$.
12. $\text{SAT} \leq_m \text{HALT}$.
13. If $\mathbf{P} = \mathbf{NP}$, then $\text{GRAPH-ISOMORPHISM} \in \mathbf{P}$.
14. If $\mathbf{NP} \neq \mathbf{coNP}$, then $\text{SAT} \notin \mathbf{coNP}$.
15. Every function $f: \{0, 1\}^n \rightarrow \{0, 1\}$ can be computed by a CNF with $O(2^n)$ clauses.

For each box below, write one of the following symbols:

- **T** if the statement is known to be true.
- **F** if the statement is false *or not known to be true*. E.g., both $\mathbf{P} = \mathbf{NP}$ and $\mathbf{P} \neq \mathbf{NP}$ should be marked **F**.
- or leave the box empty.

A correct **T/F** answer is worth +1 point, an incorrect answer is worth −1 point, and an empty answer is worth 0 points.

1.	F
2.	T
3.	F
4.	F
5.	F

6.	T
7.	T
8.	T
9.	F
10.	T

11.	F
12.	T
13.	T
14.	T
15.	T

2 (15 pts) Pumping Lemma.

2a Write down the statement of the Pumping Lemma. (*Only the statement, not its proof!*)

2b For a string $w = w_1w_2 \cdots w_n \in \{0,1\}^n$ define its *reverse* $w^{\mathcal{R}}$ as the string written backwards, that is, $w^{\mathcal{R}} = w_nw_{n-1} \cdots w_1$. Show that the following language is not regular:

$$L = \{w \in \{0,1\}^* : w^{\mathcal{R}} = w\}.$$

(A word w such that $w^{\mathcal{R}} = w$ is called a palindrome.)

Solution:

2a If A is a regular language, then there exists a number p (the pumping length) such that, for every string $s \in A$, of length at least p , there exists a division of s into three pieces $s = xyz$ such that,

- $\forall i \geq 0, xy^iz \in A$
- $|y| \geq 1$
- $|xy| \leq p$

2b Assume the given L is regular, then there exists p a pumping length. Consider $w = 0^n110^n$. Clearly $w = w^{\mathcal{R}} \implies w \in L$. Also $|w| \geq p$. Therefore, from the pumping lemma, there exists x, y, z satisfying the constraints of the pumping lemma such that $w = xyz$.

Since $|xy| \leq p$, this means x and y are completely contained within the 0^p prefix of w . This implies $|y| \leq p$, and since we also know that $|y| \geq 1$, we can state that $y = 0^k$ for $k \in [1, p]$.

Consider $xy^2z = 0^{p+k}110^p$. $xy^2z \in L$ only if $p+k = p$ i.e $k = 0$. This is a contradiction since $k \in [1, p]$.

Hence L is not regular.

Some grading guidelines:

2a (-1 pt) Ordering of the conditions in the pumping lemma.

2b (-1 pt), Directly using $y = 0^k$ rather than deducing it must be of that form and in fact we need to show that no division $w = xyz$ must exist that can be pumped.

(-2 pt) Choosing a specific decomposition instead of treating x, y, z as arbitrary, however encompassing all the $y = 0^k$ decompositions.

(-3 pt) Choosing a very specific decomposition like $y = 0$.

(-5 pt) Choosing a wrong word w and choosing a specific decomposition that fails to be pumped.

(-5 pt) Choosing a non palindrome word w .

- 3 (15 pts) **Regular languages.** For any language $A \subseteq \{0, 1\}^*$, define $\text{DROP}(A)$ to be the language containing all strings that can be obtained by removing one bit from a string in A . That is,

$$\text{DROP}(A) = \{xy : xby \in A \text{ where } b \in \{0, 1\}\}.$$

Show that if A is regular, then so is $\text{DROP}(A)$.

(Here it suffices that you describe clearly how to construct a DFA/NFA for $\text{DROP}(A)$. You do **not** need to formally prove the correctness of your construction.)

Solution:

Let $D_A = (Q, \Sigma, \delta, q_0, F)$ be the DFA that recognizes A . We define $D'_A = (Q', \Sigma', \delta', q'_0, F')$ and $D''_A = (Q'', \Sigma'', \delta'', q''_0, F'')$, two copies of D_A . We use these copies to build an NFA that recognizes $\text{DROP}(A)$.

Let $N = (Q_N, \Sigma_N, \delta_N, q_{0,N}, F_N)$ be defined as follows:

$$Q_N = Q' \cup Q''$$

$$\Sigma_N = \Sigma' \cup \{\epsilon\} = \Sigma'' \cup \{\epsilon\}$$

$$q_{0,N} = q'_0$$

$$F_N = F''$$

$$\delta_N(q, \sigma) = \begin{cases} \delta'(q, \sigma), & \text{if } q \in Q' \text{ and } \sigma \in \{0, 1\}, \\ \delta''(q, \sigma), & \text{if } q \in Q'' \text{ and } \sigma \in \{0, 1\}, \\ \delta''(q'', \sigma), & \text{if } q \in Q' \text{ and } \sigma = \epsilon. \end{cases}$$

where, in the last line, q'' is the state of D''_A corresponding to q . The starting state of N is the starting state of D'_A , the set of accepting states is the one of D''_A . These choices guarantee that one - and only one - bit is dropped.

Example:

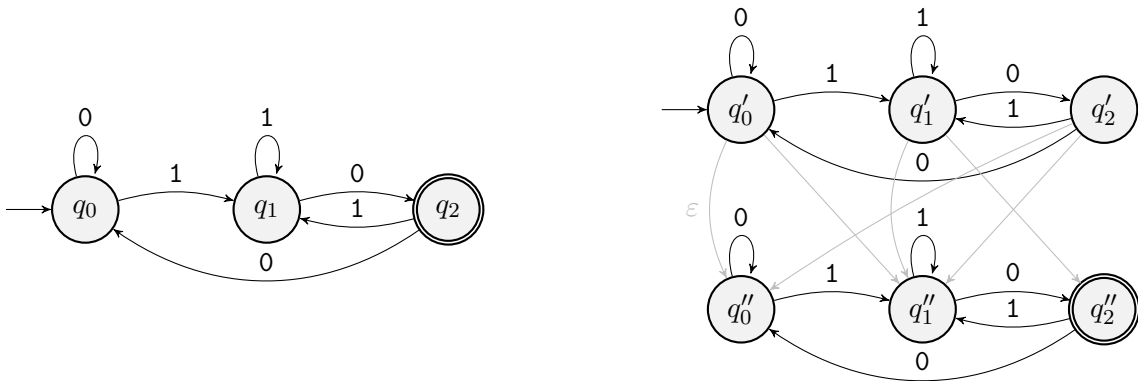


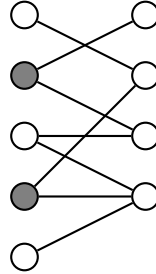
Figure 1. D_A (left) and N (right). The gray arrows in N are the new ϵ transitions.

Common mistakes and grading scheme:

- Unspecified $q_{0,N}$, wrong F_N [-2 point].
- Self-transitions not considered [-3 points].

- 0/1-transitions to corresponding state in D_A'' (add one bit instead of dropping it) [-6 points].
- D_A duplicated $2|Q|$ times, a different transition becomes an ε one in each copy [-8 points].
- D_A not duplicated, ε -transitions added without modification of D_A structure [-10 points].
- Definition of N based on input string xy [-11 points].
- Cut/copy/paste parts of D_A assuming ordered states (no "backward loops") [-11 points].
- Assumption that A is finite [-13 points].

- 4 (20 pts) **NP-completeness.** Let $G = (L \cup R, E)$ be a bipartite graph with left vertices L , right vertices R (where $L \cap R = \emptyset$), and where edges exist only between left and right vertices, that is, $E \subseteq L \times R$. We say that a subset $D \subseteq L$ is a *left-dominating set* iff for every right vertex $v \in R$ there is some $u \in D$ such that $(u, v) \in E$. For example, the highlighted vertices in the bipartite graph below form a left-dominating set of size 2.



Show that the following problem is **NP**-complete:

$$\text{LEFTDOM} = \{\langle G, k \rangle : G \text{ is a bipartite graph with a left-dominating set of size } k\}.$$

(In your proof, you may assume the **NP**-completeness of any of the problems discussed in the lectures, but **not** exercises/homework. This includes SAT, INDEPENDENT-SET, CLIQUE, VERTEX-COVER, SET-COVER, SUBSET-SUM, etc. Make sure to prove that your reduction is correct!)

Solution: We first show that $\text{LEFTDOM} \in \mathbf{NP}$. Given a bipartite graph $G = (L \cup R, E)$, a positive integer k , and a certificate C , the nondeterministic algorithm checks:

- (i) The certificate C encodes a subset of vertices $D \subseteq L$ of size k .
- (ii) For any $v \in R$, there exists $u \in D$ such that $(u, v) \in E$.

And the algorithm accepts if both condition holds. In other words, the algorithm accepts iff. C encodes a left-dominating set of size k . Thus the algorithm indeed computes LEFTDOM . Moreover, the algorithm runs in polynomial time. Hence $\text{LEFTDOM} \in \mathbf{NP}$.

Then we show that LEFTDOM is **NP**-hard by reducing SET-COVER to it. Given a SET-COVER instance $\langle \mathcal{F}, S, k \rangle$, where S is the ground set and $\mathcal{F}$ is a collection of subsets of S , we construct a LEFT-DOM instance $g(\langle \mathcal{F}, S, k \rangle) = \langle G = (L \cup R, E), k' \rangle$ as follows:

- (i) Each vertex in L corresponds to a set $F \in \mathcal{F}$, i.e. $L = \{v_F \mid F \in \mathcal{F}\}$.
- (ii) Each vertex in R corresponds to an element $s \in S$, i.e. $R = \{v_s \mid s \in S\}$.
- (iii) $E = \{(v_F, v_s) \mid F \in \mathcal{F}, s \in F\}$.
- (iv) $k' = k$.

It is easy to see that the above reduction runs in polynomial time. It remains to show $\langle \mathcal{F}, S, k \rangle \in \text{SET-COVER} \iff g(\langle \mathcal{F}, S, k \rangle) = \langle G = (L \cup R, E), k' = k \rangle \in \text{LEFT-DOM}$.

$\implies$: If $\mathcal{F}' \subseteq \mathcal{F}$ is a set cover for S of size k . Then every $s \in S$ is contained in some $F \in \mathcal{F}'$, which implies $(v_F, v_s) \in E$. Thus $\{v_F : F \in \mathcal{F}'\}$ is a left-dominating set for G of size k .

$\impliedby$: If D is a left-dominating set for G of size k . Then for every $v_s \in R$, there exists some $v_F \in D$ such that $(v_F, v_s) \in E$, which implies that $s \in F$. Thus $\{F : v_F \in D\}$ is a set cover for S of size k .

Hence LEFT-DOM is **NP**-hard.

In conclusion, LEFT-DOM is **NP**-complete.

Remark: SET-COVER generalizes VERTEX-COVER as the set of edges can be seen as the ground set, and each vertex can be seen as a set of incident edges. Hence one can reduce VERTEX-COVER to LEFTDOM in the same way. It is also not complicated to reduce SAT to LEFTDOM, where the left side contains literals (variables and their negations) and the right sides contains disjunctions. If a literal is contained in a disjunction, we add an edge between their corresponding vertices. Moreover, we need to add all the disjunctions with the form $x \vee \neg x$ to make sure together with the constraint $k = n$, exactly one of x and $\neg x$ is chosen.

Grading Scheme:

- (1) Proving LEFTDOM is **NP** is worth **5 points** and proving **NP** hardness is worth **15 points**.
- (2) I deduct **1 point** for answers that do not emphasize the certificate is a set **of size** k .
- (3) Many people forgot the definition of VERTEX-COVER and were actually reducing DOMINATING SET to LEFTDOM, which is not allowed in this problem. In this case, I deduct **4 points** if the reduction and the proof is correct.
- (4) However, some people are even not working on dominating set (they do not connect v_L and v_R). But it is still a reduction from an **NP**-complete problem to LEFTDOM. In this case, I deduct **8 points** if the proof is correct.
- (5) If the reduction does not work, I will give **[2,5] points out of 15**. The score depends on the difficulty to fix the solution, the writing and the proof structure. **3 points** is typical.

5 (20 pts) **Undecidability.** Recall the notation $w^{\mathcal{R}}$ from 2b. Define

$$T = \{\langle M \rangle : M \text{ is a TM such that for all } w, M \text{ accepts } w \text{ iff it accepts } w^{\mathcal{R}}\}.$$

Show that both T and $\overline{T}$ are unrecognisable.

Solution:

Here is a reduction $f : \overline{\text{HALT}} \rightarrow T$ where $M' = f(\langle M, w \rangle)$ is such that:

On input $\langle x \rangle$:

1. Runs M on w .
2. If $x = 01$ then accept.

f is a computable function.

We show $\langle M, w \rangle \in \overline{\text{HALT}}$ if and only if $\langle M' \rangle \in T$:

- If $\langle M, w \rangle \in \overline{\text{HALT}}$ then $L(M') = \emptyset$. Thus, $\langle M' \rangle \in T$.
- If $\langle M, w \rangle \notin \overline{\text{HALT}}$ then $L(M') = \{01\}$. Thus, $\langle M' \rangle \notin T$.

To show $\overline{\text{HALT}} \leq_m \overline{T}$ it is sufficient to show $\text{HALT} \leq_m T$. Here is a reduction $g : \text{HALT} \rightarrow T$ where $M' = g(\langle M, w \rangle)$ is such that:

On input $\langle x \rangle$:

1. If $x = 01$ then accept.
2. Runs M on w .
3. If $x = 10$ then accept.

g is a computable function.

We show $\langle M, w \rangle \in \text{HALT}$ if and only if $\langle M' \rangle \in T$:

- If $\langle M, w \rangle \in \text{HALT}$ then $L(M') = \{10, 01\}$. Thus, $\langle M' \rangle \in T$.
- If $\langle M, w \rangle \notin \text{HALT}$ then $L(M') = \{01\}$. Thus, $\langle M' \rangle \notin T$.

Grading scheme:

- 10 points for T
- 10 points for $\overline{T}$

Within each: 5 points for each direction.

Typical mistakes:

- 0 - no answer, only sketch

- 5 - some argument, no correctness or utterly wrong
- 10 - correctness argument but flawed, i.e. because didn't realise modification needed
- 15 - correctness arguments makes sense, i.e. realised some modification is needed
- 20 - very good, modification realised and carried out correctly

- 6 (15 pts) **CNF Equivalence.** We say that two CNF formulas φ and ψ , both defined over the same set of n variables $x = (x_1, \dots, x_n)$, are *equivalent* if they compute the same boolean function, that is, $\varphi(x) = \psi(x)$ for all $x \in \{0, 1\}^n$. Consider the language

$$\text{CNF-EQ} = \{\langle \varphi, \psi \rangle : \varphi \text{ and } \psi \text{ are a pair of equivalent CNFs}\}.$$

Classify this problem into as small a complexity class as possible: is it in **P**, **NP**, **coNP**, or merely decidable? Is it complete for any complexity class? Justify your answer with a proof.

(Note that, in general, for any class of languages **C**, we say a language A is **C**-complete iff $A \in \mathbf{C}$ and we have $L \leq_p A$ for all $L \in \mathbf{C}$.)

Solution: We show that the language CNF-EQ is **coNP**-complete.

First of all, $\text{CNF-EQ} \in \mathbf{coNP}$ because $\overline{\text{CNF-EQ}}$ is the language of all pairs of functions (φ, ψ) that are not equivalent and non-equivalency can be certified by exhibiting an assignment $x \in \{0, 1\}^n$ with $\varphi(x) \neq \psi(x)$. Verification takes polynomial time.

We further show that the **coNP**-complete problem $\overline{\text{SAT}}$ reduces to CNF-EQ. Indeed, let f be the poly-time computable function that takes a formula φ and returns the CNF-EQ instance $(\varphi, x_1 \wedge \neg x_1)$. Then, for any formula φ :

$$\varphi \in \overline{\text{SAT}} \iff \varphi = 0 \iff \varphi = x_1 \wedge \neg x_1 \iff f(\varphi) \in \text{CNF-EQ}$$

Finally, we know that **NP** and **coNP** are thought to be unequal and no **coNP**-complete problem can be in **NP**. Thus, most likely $\text{CNF-EQ} \notin \mathbf{P}$ and even $\text{CNF-EQ} \notin \mathbf{NP}$.

Grading scheme for the problem 6. Solution as presented above consists of three parts, and they were graded as follows:

- Stating that the problem is in **coNP** – **3 points**, proving it – **3 points**.
- Claiming that the problem is **coNP**-complete – **3 points**, proving it – **3 points**.
- The conclusion that **coNP** is indeed the smallest complexity class most probably, because of **coNP** $\neq$ **NP** – **3 points**.

It was also possible to receive partial points if they WERE NOT part of an argument for the points received from above.

- Stating that the problem is decidable – **1 point**, and giving the correct verifier – **1 point**.
- Reasonable attempts to show that the problem is not in **P** – **1 point** and it's not in **NP** – **1 point**.
- It is mentioned that SAT is useful for defining the complexity class – **1 point**, and given explanations that it is useful because it is **coNP**-complete problem and we can find the reduction from it to CNF-EQ – **3 points**.

Finally, there were several common mistakes, approaches:

- Trying to show the reduction from CNF-EQ to $\overline{\text{SAT}}$ and not the other way around.
- Showing that the verifier of CNF-EQ is polytime.
- Trying to show the reduction of SAT to CNF-EQ.
- Just writing many facts and claims all together without any justification.

Also, for small or big mistakes one or two points were subtracted.